

NanoCore16

MODULE IMPLEMENTATION SPECIFICATION

Educational Processor — ComputEdge Technologies

16-bit

Instructions

8-bit

Datapath

16

Registers

6

Modules



SYSTEM PARAMETERS & ISA OVERVIEW

System Parameters

Instruction Width	16 bits
Data Width	8 bits
Address Width	16 bits (PC)
Register Count	16 (R0-R15)
Clock	Single rising-edge
Reset	Active-low async

Instruction Format [15:0]

OPCODE	Rd	RS1	RS2/IMM
[15:12]	[11:8]	[7:4]	[3:0]

4 bits each — fixed-width encoding

R-type

Register ops: ADD, SUB, AND, OR, XOR, INV, MOV, CMP

I-type

Immediate: ADDI, SHL, SHR, LDI, JMP

J-type

Branch: BEQ, BNE

NOP

No operation — PC+1 only

COMPLETE INSTRUCTION SET — ALL 16 OPCODES

OPCODE	Hex	Mnemonic	Type	Operation	Description
0000	0x0	ADD	R	$Rd \leftarrow RS1 + RS2$	Add two registers
0001	0x1	SUB	R	$Rd \leftarrow RS1 - RS2$	Subtract RS2 from RS1
0010	0x2	ADDI	I	$Rd \leftarrow RS1 + \text{sign_ext}(\text{IMM4})$	Add register + signed imm
0011	0x3	AND	R	$Rd \leftarrow RS1 \& RS2$	Bitwise AND
0100	0x4	OR	R	$Rd \leftarrow RS1 RS2$	Bitwise OR
0101	0x5	XOR	R	$Rd \leftarrow RS1 \wedge RS2$	Bitwise XOR
0110	0x6	INV	R	$Rd \leftarrow \sim RS1$	Bitwise NOT (RS2 ignored)
0111	0x7	SHL	I	$Rd \leftarrow RS1 \ll \text{IMM4}[2:0]$	Logical shift left 0–7
1000	0x8	SHR	I	$Rd \leftarrow RS1 \gg \text{IMM4}[2:0]$	Logical shift right 0–7
1001	0x9	LDI	I	$Rd \leftarrow \text{sign_ext}(\text{IMM4})$	Load signed immediate
1010	0xA	MOV	R	$Rd \leftarrow RS1$	Copy register (RS2 ignored)
1011	0xB	CMP	R	$\text{flags} \leftarrow RS1 - RS2$	Compare — sets flags, NO writeback
1100	0xC	JMP	I	$PC \leftarrow RS1 + \text{sign_ext}(\text{IMM4})$	Unconditional jump
1101	0xD	BEQ	J	$\text{if}(\text{zero}) \text{ } PC \leftarrow PC + \text{imm}$	Branch if equal (zero flag)
1110	0xE	BNE	J	$\text{if}(!\text{zero}) \text{ } PC \leftarrow PC + \text{imm}$	Branch if not equal
1111	0xF	NOP	—	No operation	PC increments, no reg write

MODULE IMPLEMENTATION OVERVIEW



program_counter

Sequential

Holds & updates 16-bit PC. Async reset, branch/stall support.



instruction_memory

Combinational

ROM — 256×16-bit, async read via \$readmemh. No clock.



instruction_decoder

Combinational

Control unit — extracts fields, generates alu_op, reg_wr_en, branch_en.



register_file

Mixed

16×8-bit regs. Async reads, sync write. R0 hardwired = 0.



alu

Combinational

8-bit ALU: ADD,SUB,AND,OR,XOR,INV,SHL,SHR,LDI,MOV. zero & carry flags.



nanocore16_top

Top-Level

Wires all submodules. operand_b MUX, branch_target logic, 7-seg display.

program_counter

clk	in	1	System clock
rstn	in	1	Async reset (active-low)
pc_en	in	1	Enable / stall control
branch_en	in	1	Load branch_target when 1
branch_target	in	16	Jump destination address
pc_out	out	16	Current PC value

⚡ Behaviour

```
rstn=0 → pc_out ← 0x0000 (async)
pc_en=1, branch_en=0 → pc_out ← pc_out + 1
pc_en=1, branch_en=1 → pc_out ← branch_target
pc_en=0 → pc_out holds (stall)
```

instruction_memory

addr	in	16	From PC — only [7:0] used
instr	out	16	Instruction word (combinational)

RTL Implementation

```
logic [15:0] mem [0:255];

initial $readmemh("program.hex", mem);

assign instr = mem[addr[7:0]];

// No always block — purely combinational
```

Key Points

- 256-word × 16-bit ROM depth
- addr[7:0] only — upper bits ignored
- Out-of-range → returns 16'hF000 (NOP)
- No clock, no reset — zero-latency read

MODULE 3 · INSTRUCTION DECODER (Control Unit)

Port Table

instr	in	16	Full 16-bit instruction word
zero_flag	in	1	From ALU — used by BEQ/BNE
opcode	out	4	instr[15:12]
rd_addr	out	4	instr[11:8]
rs1_addr	out	4	instr[7:4]
rs2_addr	out	4	instr[3:0]
imm	out	8	Sign-ext: {{4{instr[3]}},instr[3:0]}
alu_op	out	4	ALU operation select
reg_wr_en	out	1	Write enable for register file
imm_sel	out	1	1 = use immediate; 0 = use rs2
branch_en	out	1	1 = take branch; 0 = PC+1

Control Signal Truth Table (selected)

Opcode	alu_op	wr_en	imm_sel	branch_en
ADD (0000)	0000	1	0	0
SUB (0001)	0001	1	0	0
ADDI (0010)	0000	1	1	0
SHL (0111)	0111	1	1	0
LDI (1001)	1001	1	1	0
CMP (1011)	0001	0	0	0
JMP (1100)	xxxx	0	x	1
BEQ (1101)	xxxx	0	x	zero
BNE (1110)	xxxx	0	x	~zero

⚠ CMP: reg_wr_en MUST be 0 — result discarded, flags only!

register_file

<code>clk</code>	1	Write on posedge clk
<code>rstn</code>	1	Async reset — all regs → 0x00
<code>wr_en</code>	1	Write enable
<code>rd_addr</code>	4	Destination reg index
<code>rs1_addr</code>	4	Source reg 1 index
<code>rs2_addr</code>	4	Source reg 2 index
<code>wr_data</code>	8	Data to write
<code>rs1_data</code>	8	Async read port 1
<code>rs2_data</code>	8	Async read port 2

R0 is hardwired to 0x00 — guard all writes with (rd_addr != 4'h0)

alu

op	Mnem.	Result	Flags
0000	ADD	a + b	zero+carry
0001	SUB	a - b	zero+carry
0011	AND	a & b	zero
0100	OR	a b	zero
0101	XOR	a ^ b	zero
0110	INV	~a	zero
0111	SHL	a << b[2:0]	zero
1000	SHR	a >> b[2:0]	zero
1001	PASS_B	b (LDI)	zero
1010	PASS_A	a (MOV)	zero

carry_flag = 9th bit of ADD/SUB only | zero_flag = (result == 8'h00)

SINGLE-CYCLE TIMING MODEL & TOP-LEVEL CONNECTIONS

Cycle Timing (posedge N → posedge N+1)

- 1 `PC → addr → IMEM → instr[15:0]`
- 2 `instr → DECODER → control signals + field addresses`
- 3 `rs1_addr, rs2_addr → REGFILE → rs1_data, rs2_data`
- 4 `imm_sel MUX → operand_b selected`
- 5 `rs1_data, operand_b, alu_op → ALU → alu_result, flags`
- 6 `branch_target computed, branch_en determined`

At posedge N+1: `PC ← pc_next` | `reg[rd_addr] ← alu_result` (if `reg_wr_en=1`)

Golden Rules for Implementation

- Combinational modules → `assign` / `always_comb` | NEVER `always_ff`
- Sequential modules → `always_ff` @(posedge `clk` or negedge `rstn`) | NEVER blocking =
- Set all outputs to safe DEFAULTS at top of every `always_comb` block (prevents latches)
- Test each module standalone before connecting in top-level | R0 always = 0x00

VERIFICATION PLAN & TEST PROGRAMS

Module-Level Tests

- **Reset test** rstn=0 → PC=0, all regs=0x00
- **ADD/SUB/ADDI** Load via LDI, compute, verify Rd
- **AND/OR/XOR** Use patterns: 0xAA, 0x55, 0xFF
- **INV** ~0xAA = 0x55, RS2 has no effect
- **SHL/SHR** Shift 0,1,7; shift by 8 = 0
- **LDI pos/neg** LDI R1,5→0x05 | LDI R1,-1→0xFF
- **MOV** MOV R2,R1 → R2 = R1 value
- **CMP** Equal values → zero=1, Rd UNCHANGED
- **JMP** PC ← RS1+imm (not RS1 alone)
- **BEQ/BNE** Taken & not-taken conditions

Test 1 — Basic Arithmetic

```
LDI R1, 5 ; 9105 → R1=0x05
LDI R2, 3 ; 9203 → R2=0x03
ADD R3,R1,R2; 0312 → R3=0x08
CMP R0,R3,R0; B030 → zero=0
```

→ **R3=0x08, zero=0, carry=0**

Test 2 — Branch Loop (count 3→0)

```
LDI R1, 3 ; 9103
LDI R2, -1 ; 920F → R2=0xFF
ADD R1,R1,R2; 0112 ← loop
CMP R0,R1,R0; B010
BNE -2 ; E00E
NOP ; F000
```

→ **Loop runs 3x, R1=0x00, zero=1**

Test 3 — Logic & Shift

```
LDI R1, 0xF ; 910F→R1=0x0F
INV R2,R1 ; 6210→R2=0xF0
SHL R3,R1,2 ; 7312→R3=0x3C
```

IMPLEMENTATION CHECKLISTS

program_counter

- ☐ always_ff @(posedge clk or negedge rstn)
- ☐ rstn=0 → pc_out ← 16'h0000
- ☐ pc_en=1, branch_en=0 → pc_out ← pc_out + 1
- ☐ pc_en=1, branch_en=1 → pc_out ← branch_target
- ☐ pc_en=0 → hold (stall)

instruction_decoder

- ☐ 4 assign statements for field extraction
- ☐ Sign-ext: imm = {{4[instr[3]]},instr[3:0]}
- ☐ always_comb — defaults set BEFORE case
- ☐ CMP: reg_wr_en=0 (no writeback!)
- ☐ BEQ: branch_en=zero_flag | BNE: ~zero_flag

alu

- ☐ 9-bit temp wire for carry: {1'b0,a} ± b
- ☐ always_comb — safe defaults first
- ☐ All 10 ops + default case
- ☐ carry_flag = temp[8] for ADD/SUB only
- ☐ zero_flag = (alu_result == 8'h00)

instruction_memory

- ☐ logic [15:0] mem [0:255];
- ☐ \$readmemh("program.hex", mem)
- ☐ assign instr = mem[addr[7:0]];
- ☐ No always block, no clock, no reset
- ☐ Out-of-range → 16'hF000 (NOP)

register_file

- ☐ logic [7:0] regs [0:15];
- ☐ Async read: two assign statements
- ☐ always_ff — non-blocking <= for writes
- ☐ R0 guard: if (rd_addr != 4'h0)
- ☐ Reset: all regs ← 8'h00

nanocore16_top

- ☐ Instantiate all 6 submodules
- ☐ operand_b MUX: imm_sel?imm:rs2_data
- ☐ branch_target: JMP=rs1+imm, BEQ/BNE=PC+imm
- ☐ seven_seg: data_in={alu_result,pc_out[7:0]}
- ☐ No new logic — wiring only